Mayan SVM

Security Assessment

December 12th, 2025 — Prepared by OtterSec

Akash Gurugunti	sud0u53r.ak@osec.io
Gabriel Ottoboni	ottoboni@osec.io
Nicholas R. Putra	nicholas@osec.io
Robert Chen	r@osec.io

Table of Contents

Appendices

01 — Executive Summary

Overview

Mayan Finance engaged OtterSec to assess the `swift` program. This assessment was conducted between November 10th and November 17th, 2025. A follow-up audit was conducted between January 23rd and January 25th, 2026. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 5 findings throughout this audit engagement.

In particular, we identified a vulnerability concerning the unlock instruction that transfers a computed net amount instead of the actual remaining balance, which may leave leftover locked tokens after partial failures and block account closure (OS-MSM-ADV-00).

Additionally, the flag for checking if the configuration is enabled is never enforced, so the program cannot be paused even during emergencies (OS-MSM-ADV-01).

We also made recommendations regarding modifications to the codebase for improved functionality and efficiency (OS-MSM-SUG-01). We further suggested ensuring adherence to best coding practices and addressing instances of redundant or unutilized code (OS-MSM-SUG-02). Lastly, we advised wrapping the transfer and deposit fee in a single atomic block to ensure FeeManager state is updated only when the transfer succeeds (OS-MSM-SUG-00).

02 — Scope

The source code was delivered to us in Git repositories at github.com/mayan-finance/anchor and github.com/mayan-finance/swap-bridge. This audit was performed against PR#4 and PR#23, respectively. The follow-up audit was performed against PR#8.

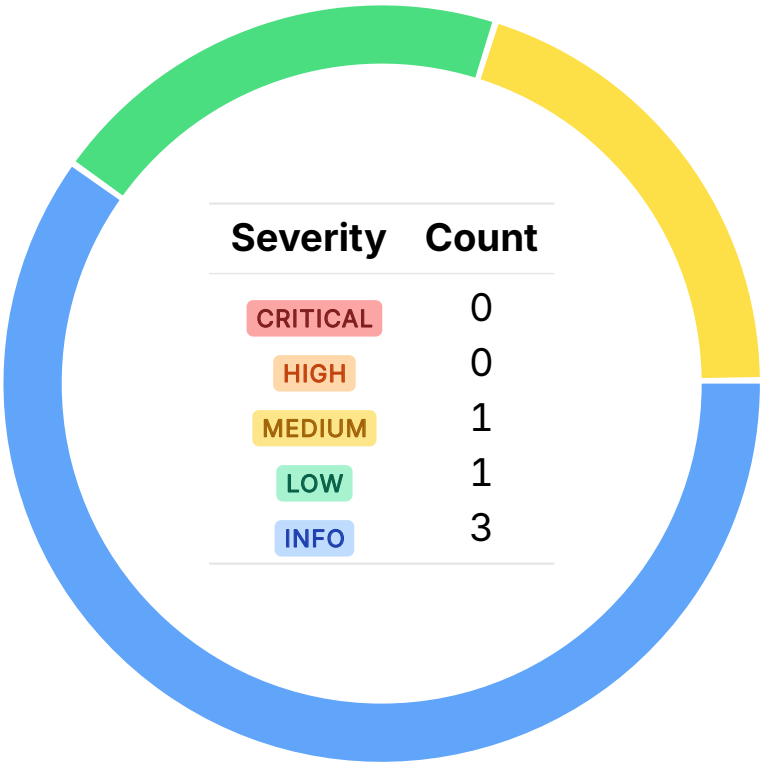
A brief description of the program is as follows:

Name	Description
swift	A cross-chain auction and settlement system that uses Wormhole messaging to coordinate price discovery, bond locking, fulfillment, and refunds across different blockchains. The audit specifically focuses on the updates made to SVM implementation to expand Swift V2 to support the Fogo chain.

03 — Findings

Overall, we reported 5 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	1
LOW	1
INFO	3

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-MSM-ADV-00	MEDIUM	RESOLVED	The `Unlock` instruction transfers a computed `amount_net` instead of the actual remaining balance, which may leave leftover locked tokens after partial failures and block account closure.
OS-MSM-ADV-01	LOW	RESOLVED	The `Config.enabled` flag is never enforced, so the program cannot be paused even during emergencies.

Improper Unlock Amount Handling MEDIUM OS-MSM-ADV-00

Description

The `unlock` instruction transfers only `amount_net` (the expected post-fee value) to `unlock_receiver_acc` rather than the actual remaining balance in `state_from_acc`. If prior unlock attempts soft-fail, the associated token account may retain leftover tokens. These residual tokens also prevent `close_account` from succeeding, resulting in errors when closing `state_from_acc`.

>_ *programs/swift/src/processor/unlock.rs* RUST

```rust
pub fn handle_unlock_order<'info>([...]) -> Result<()> {
    [...]
    token_interface::transfer_checked(
        CpiContext::new_with_signer(
            token_program.to_account_info(),
            token_interface::TransferChecked {
                from: state_from_acc.to_account_info(),
                to: unlock_receiver_acc.to_account_info(),
                authority: state.to_account_info(),
                mint: mint_from.to_account_info(),
            },
            &[state_seeds],
        ),
        amount_net,
        mint_from.decimals,
    )?;
    [...]
    Ok(())
}
```

Remediation

Transfer `state_from_acc.amount` instead of `amount_net`.

Patch

Fixed in 7eb7bd5.

Absence of Global Pause Enforcement LOW OS-MSM-ADV-01

Description

The `Config.enabled` flag is never checked in any instruction, implying the program cannot actually be paused even if the flag is set to false. This renders all critical flows fully operational during emergencies. If this flag was intended as a safety switch, its absence from instruction validations renders it ineffective and exposes the system to continued execution during incidents.

Remediation

Validate `Config.enabled` across all instructions if it is meant to act as a pausing mechanism.

Patch

Fixed in 7eb7bd5.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MSM-SUG-00	Separating `depositFee` and the token transfer into different `try/catch` blocks may result in `FeeManager` balances to increase even when no tokens were actually transferred.
OS-MSM-SUG-01	Recommendation for updating the codebase to improve functionality and overall efficiency.
OS-MSM-SUG-02	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.

Atomic Fee Accounting

OS-MSM-SUG-00

Description

In `unlockOrder`, executing `depositFee` and the token transfer in separate `try/catch` blocks may result in inconsistent accounting. `depositFee` may update `FeeManager` balances even when the underlying token transfer fails. Because the transfer failure is silently caught, the `FeeManager` may record fees it never actually received.

Remediation

Wrap the transfer and `depositFee` in a single atomic block to ensure `FeeManager` state is updated only when the transfer succeeds.

Patch

The Mayan team acknowledged the issue.

Code Refactoring OS-MSM-SUG-01

Description

1. The `Cancel` instruction should first verify that the auction is not already cancelled. Without this, the initiator may cancel the same auction multiple times, resulting in duplicate Wormhole messages.
2. `PostAuctionShim` instruction does not verify whether the auction was cancelled or whether a valid winner exists that could allow posting of fulfillment messages for invalid or no-bid auctions. Check that the auction is not cancelled and that the winner is non-null.
3. The current logic in `RequestChangeAdmin` instruction prevents resetting `next_admin`, implying that once an ownership transfer is initiated, it cannot be revoked easily. Allowing `next_admin` to be set to the zero address will enable an owner to revoke a pending admin change easily if it was set incorrectly or compromised.
4. `RefundAuctionIr` performs emitter validation in two places. `verify_fast_refund_vaa_shim` checks against the hardcoded `FAST_REFUNDER_EMITTER_CHAIN_ID`, while the handler separately checks `emitter_chain == config.fast_refunder_chain_id`. If these values diverge, a valid VAA may pass shim verification but fail the configuration check. This introduces a configuration inconsistency risk that may block legitimate refunds. Ensure to validate the `emitter_chain` once in `RefundAuctionIr`.

>_ programs/swift/src/processor/refund_auction_ir.rs RUST

```rust
pub fn handle_refund_auction_ir(
    ctx: Context<RefundAuctionIr>,
    order_info: OrderInfo,
    wormhole_guardian_set_bump: u8,
    vaa_bytes: Vec<u8>,
    try_to_close: bool,
) -> Result<()> {
    [...]
    // shim path
    let (payload, emitter_chain) = verify_fast_refund_vaa_shim(
        &ctx.accounts.guardian_set_info,
        &ctx.accounts.guardian_signatures_info,
        wormhole_guardian_set_bump,
        &vaa_bytes,
    )?;
    [...]
    require!(emitter_chain == ctx.accounts.config.fast_refunder_chain_id,
        ↪  SwiftError::InvalidFastRefunderChainId);
    [...]
}
```

Remediation

Incorporate the above refactors.

Code Maturity OS-MSM-SUG-02

Description

1. `ChangeConfig` and `RequestChangeAdmin` instructions both load and validate the same core account set. Maintaining separate but nearly identical context structures results in unnecessary duplication. Re-utilize a single shared context to reduce code redundancy.
2. In `PostChangeRefundVerifier`, the message logging the sequence number of the current post message incorrectly states *"cancel message seq"* even though the instruction does not perform a cancel action, which renders the message misleading. Similar message is utilized in `RescueFunds`.

>_ *programs/swift-auction/src/state/auction_state.rs* RUST

```rust
pub fn handle_post_change_refund_verifier(
    ctx: Context<PostChangeRefundVerifier>,
    params: PostChangeRefundVerifierParams,
) -> Result<()> {
    [...]
    msg!("cancel message seq: {}", next_sequence.saturating_sub(1));
    Ok(())
}
```

3. Several log messages and error strings still hard-code the term *"solana"*, even though the code may run on the Fogo chain in the current context. Update these strings to utilize chain-neutral terminology due to the chain-agnostic environment.
4. `AuctionBond::active_bond_index` is just a list index, so utilizing `u64` is unnecessary. It will be more appropriate to switch to `u32` for its intended range. Additionally, in `AuctionState`, the `override_from` field is not utilized anywhere and may be removed.

>_ *programs/swift-auction/src/state/auction_state.rs* RUST

```rust
#[derive(Debug, InitSpace, Clone, Copy, AnchorSerialize, AnchorDeserialize)]
pub struct AuctionBond {
    pub bond_mint_config: Pubkey,
    pub active_bond_index: u64,
    pub base_bond: u64,
    pub per_bps_bond: u64,
    pub tick_size: u64,
    pub released: bool,
}
```

5. Utilize a named constant `WH_CHAIN_ID_SUI` for `FAST_REFUNDER_EMITTER_CHAIN_ID`, including in the `initialize` instruction, instead of the literal value 21. This explicitly conveys that the value represents the Wormhole chain ID for Sui, and reduces the risk of inconsistencies if the value changes.

Remediation

Implement the above-mentioned suggestions.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.